Personal Finance and Investment Relationship Analysis

Using Neo4j Community Edition

DA3 Mini Project Report
Database category Graph Database
Submission date 20 October 2026

Student name _____

Registration number _____

Faculty name _____

Abstract

This project extends an Oracle based personal finance mini project using Neo4j, a property graph database. It retains households, users, accounts, banks, companies, investments, subscriptions, savings goals and ledger records. Explicit graph relationships enable employer and investment overlap, household company exposure and shared investment queries. A Flask application provides complete record management, an interactive graph explorer and fifteen Cypher reports.

The implementation stores monetary values as integer paise and groups ledger changes, balance adjustments and audit snapshots in atomic managed transactions. Real database integration tests verify CRUD, rollback, concurrency and persistence. Neo4j is justified for connected analysis; the comparison recognizes the strengths of relational databases for ledger integrity and reporting.

Report contents

Application and database choice; graph schema; dataset and integrity; browser implementation; Cypher reports; graph query results; visual demonstration; verification; comparison; setup and references.

Application and database choice

Problem and objectives

Personal finance information is spread across people, accounts, employers and investment holdings. The DA2 implementation uses Oracle tables, SQL reports and PL/SQL programs. DA3 implements the same domain in Neo4j and explores connections that a tabular dashboard does not show directly.

The objectives are to design a property graph, import the original fictional data, implement core operations, demonstrate technology specific queries and explain the suitability and limitations of the selected database.

Why Neo4j

Neo4j represents entities as labeled nodes, typed directed relationships and properties on both [1]. A user can belong to a household, own accounts, work for a company and hold that company's shares. Cypher expresses those connected patterns directly. The selected database is in the Graph Database category explicitly allowed by the assignment.

The implementation uses Community Edition, standard Cypher and the official Python driver. It requires no paid service, embedding model, graph plugin or external API for normal operation. Graph insights are explainable through returned paths.

Alternatives considered

Option	Suitability assessment for this project
Time series database	Useful for a workload centered on dense historical measurements; this extension prioritizes relationships across existing entities.
Vector database	Would require a semantic similarity use case and an embedding pipeline beyond the current structured finance data.
Column family database	A distributed ingestion or partition scale requirement is not established by this small classroom workload.
Distributed SQL	Could retain SQL and relational structure; it would demonstrate less of the selected relationship modeling change.
Neo4j graph database	Matches the chosen connected analysis and provides visible nodes, relationships and Cypher paths.

These are project design judgments, not general rankings or performance benchmarks. SQL can also answer the selected questions using joins.

Property graph schema

Ten business node labels retain the original conceptual entities. Share ownership becomes a relationship with a quantity property. AuditEvent supports history; DemoState supports import state and serialized application writes.

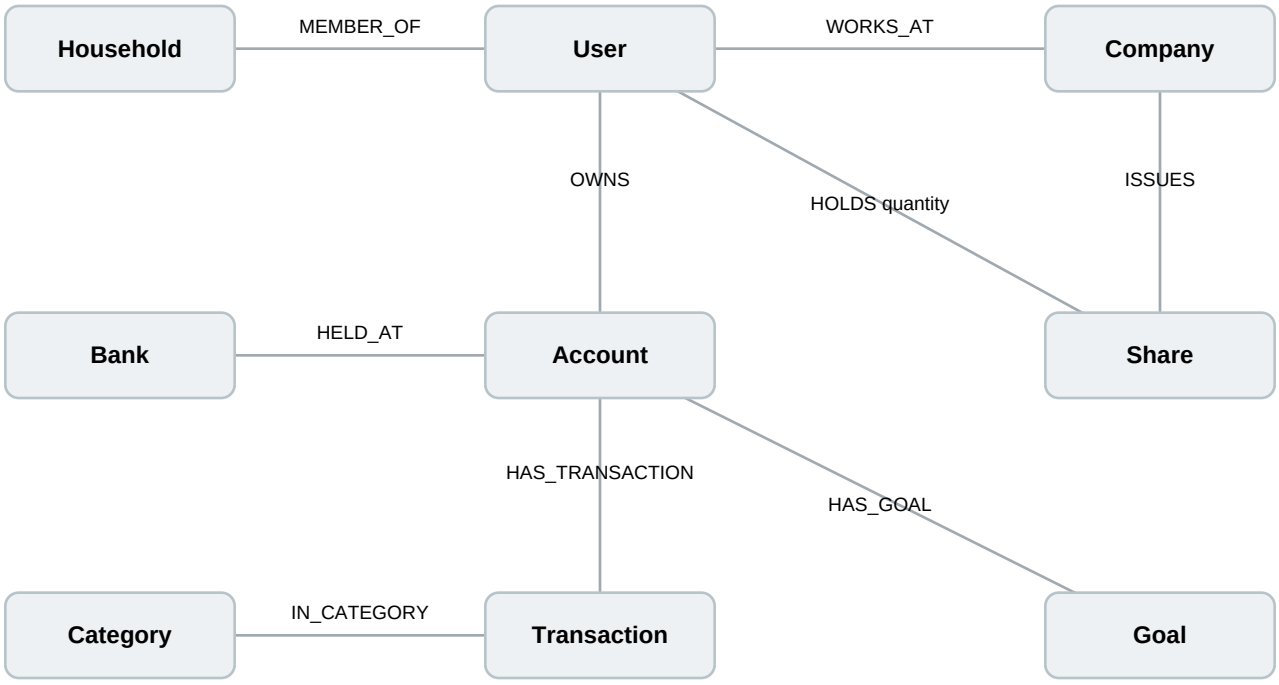


Figure 1. Main graph schema. Arrow direction is specified by the relationship definitions below. Subscription and audit records are omitted from the drawing for readability.

Relationship	Direction and meaning
MEMBER_OF and WORKS_AT	User to Household; User to Company
OWNS and HELD_AT	User or Company to Account; Account to Bank
ISSUES and HOLDS	Company to Share; User to Share, with quantity and pair key
HAS_TRANSACTION and IN_CATEGORY	Account to Transaction; Transaction or Goal to Category
HAS_GOAL and HAS_SUBSCRIPTION	Account to Goal; Account to Subscription

IDs have unique key properties. Subscription keys combine account and subscription IDs. Audit events retain transaction IDs and old/new snapshots after a transaction is deleted.

Dataset and transactional integrity

Portable fictional sample dataset

Collection	Count	Collection	Count
Households	3	Companies	4
Banks	3	Categories	9
Users	5	Accounts	7
Shares	4	Holdings relationships	6
Subscriptions	5	Goals	5
Transactions	12	Initial audit events	12

The data/sample.json fixture contains opening balances, not already adjusted balances. A single atomic import creates all records and applies the twelve sample transactions once. The resulting graph has 57 business nodes and 72 relationships. Twelve audit nodes and one state node bring the total stored node count to 70. Repeating import preserves existing edits.

Exact money and stable references

Values entered in rupees are parsed using Decimal and stored as integer paise. For example, INR 250 is stored as 25000. Native date properties retain transaction dates and dates of birth. Unique keys prevent duplicate identifiers; application validation checks required values, allowed types, ownership, references and dependent deletion.

Atomic ledger algorithm

```
BEGIN managed write transaction
  Increment DemoState revision to acquire the shared write lock
  Read old ledger entry and validate new references
  Reverse old balance effect and apply new effect as net changes
  Reject any resulting negative account balance
  Persist ledger entry and permanent audit snapshot
COMMIT together or ROLLBACK together
```

The single state lock serializes all application writes before checks or balance reads. This conservative classroom design prevents conflicting writes but limits throughput. Managed transactions are provided by the Neo4j driver [2]; locking and isolation behavior are documented by Neo4j [4].

Account 1001 reconciles as 25000 + 50000 - 4500 - 2500 - 1200 = INR 66800. All seven accounts reconcile to their opening balances and ledgers.

Browser application and core operations

The standalone Flask application runs on port 5001 and uses Waitress. A local Neo4j container stores the graph. All application queries use the official Python driver. The interface includes the overview, management screens, reports and graph explorer.

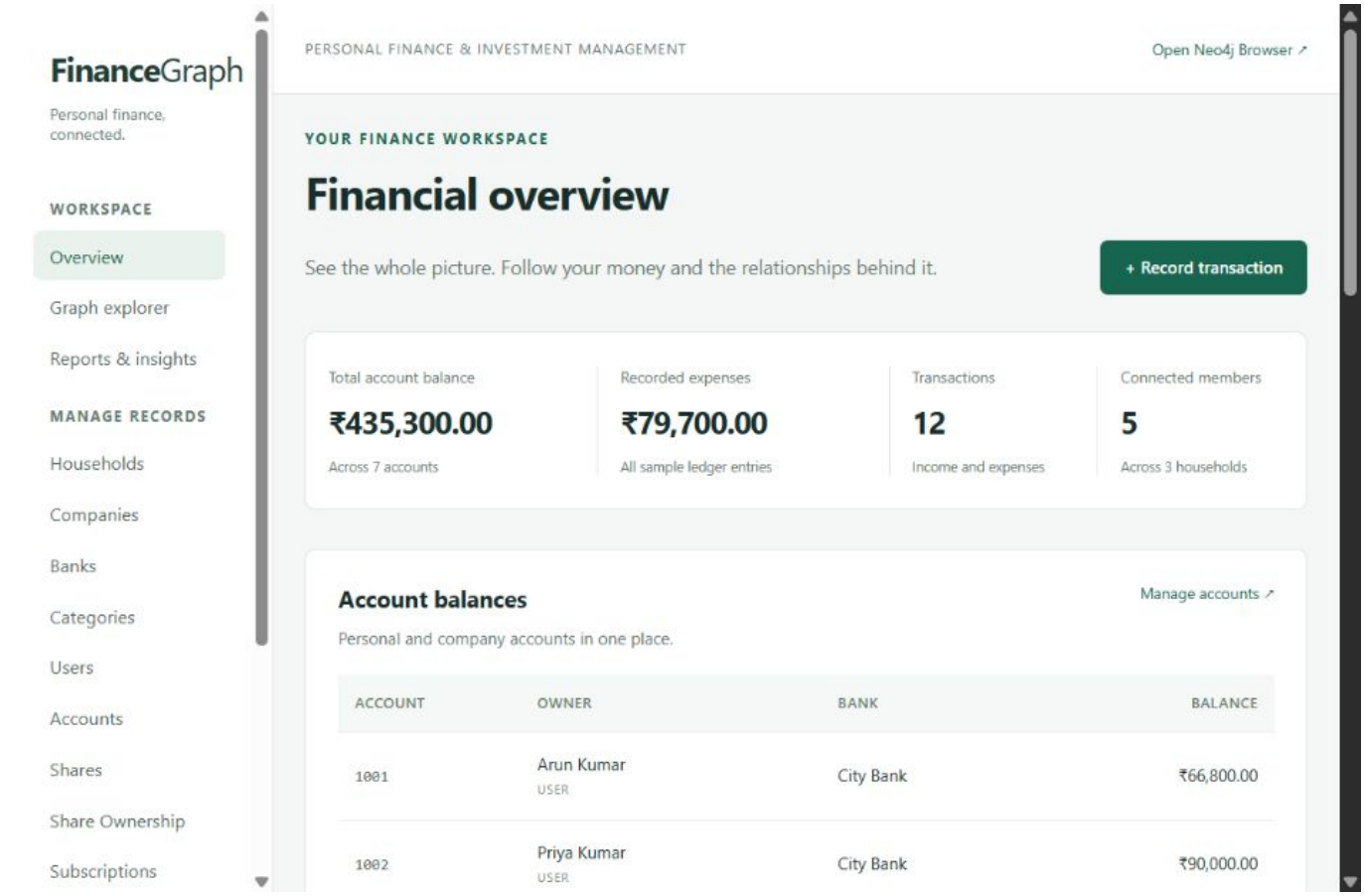


Figure 2. Actual overview screenshot. The baseline total account balance is INR 435300 and recorded expenses total INR 79700.

Implemented operations

Operation	Implemented behavior
Create and read	All eleven original business collections, including relationships and composite keys.
Update	Editable entity properties, relationships, holdings quantity and ledger values.
Delete	Restricted dependent deletion; ledger deletion reverses its effect and retains audit history.
Form protection	Server validation, parameterized queries, escaped templates and session CSRF tokens.

A real browser test submitted the new-user form and verified the success message. Its temporary user was removed after capturing evidence. Additional form screenshots are included in the submission source.

Cypher reports and financial queries

The application exposes all twelve DA2 reports rewritten in Cypher and three additional graph insights. Each report shows its query and live results, with a CSV download. The complete runnable statements are in queries.cypher.

ID	Report	Demonstrated concept
Q01	Household members and ages	Relationship match and date duration
Q02	Account owners and bank balances	Connected ownership pattern
Q03	Investment portfolio value	Relationship properties and calculation
Q04	Spending by category	Filtering and aggregate sum
Q05	Accounts above average balance	Aggregation followed by filtering
Q06	Savings goal progress	Expression and rounding
Q07	Monthly subscription cost	CASE and cycle normalization
Q08	Company employee counts	OPTIONAL MATCH includes zero
Q09	Largest expense per account	Grouped maximum and matching ties
Q10	Monthly income and expenses	Dates and conditional aggregation
Q11	Total balances by bank	Optional links and aggregation
Q12	Permanent transaction audit	History after ledger deletion
G01	Employer and investment overlap	Closed connected pattern
G02	Household company exposure	Multiple relationship traversal
G03	Shared investment connections	Common-neighbor pattern

Example category spending query

```
MATCH (t:Transaction)-[:IN_CATEGORY]->(c:Category)
WHERE t.trans_type='EXPENSE'
RETURN c.name AS category, count(t) AS transactions,
sum(t.amount_paise)/100.0 AS spent_rupees ORDER BY spent_rupees DESC
```

Actual expense totals are Salary 50000, Travel 12000, Utilities 7500, Food 7200 and Investment 3000 rupees. The Salary expense category follows the original company sample transaction; category and income/expense direction are separate fields.

Graph specific queries and results

Employer and investment overlap

```
MATCH (u:User)-[:WORKS_AT]->(c:Company)-[:ISSUES]->(s:Share)<-[h:HOLDS]-(u)
RETURN u.name AS member,c.name AS employer,s.share_id AS share_id,
h.quantity*s.market_price_paise/100.0 AS investment_rupees ORDER BY member
```

Member	Employer	Investment INR
Arun Kumar	Nexa Technologies	3,600.00
Meera Singh	GreenWave Energy	6,400.00
Priya Kumar	BlueSky Retail	2,600.00
Rahul Rao	Nexa Technologies	2,400.00

This pattern connects a user to an employer and to shares issued by that same employer. Four users match in the sample dataset. It identifies overlapping salary and investment exposure for exploration; it does not measure total financial risk.

Household company exposure

```
MATCH (h:Household)<-[:MEMBER_OF]-(u:User)-[holding:HOLDS]->(s:Share)<-[:ISSUES]-(c:Company)
RETURN h.name AS household,c.name AS company,collect(DISTINCT u.name) AS investors,
sum(holding.quantity*s.market_price_paise)/100.0 AS exposure_rupees
ORDER BY household,exposure_rupees DESC
```

Household	Company	Exposure INR
Kumar Household	BlueSky Retail	5,850.00
Kumar Household	Nexa Technologies	3,600.00
Rao Household	Nexa Technologies	5,400.00
Singh Household	GreenWave Energy	6,400.00

Household exposure follows members, their HOLDS relationships and issuing companies. The result combines Rahul and Dev's different Nexa shares into INR 5400 for Rao Household.

Graph visualization and shared investments

The explorer queries Neo4j and renders returned entities and relationships in local SVG. Users can select nodes, inspect properties, filter by household or insight, pan, zoom, fit the graph and show relationship labels. Standard Cypher graph variants also run in Neo4j Browser.

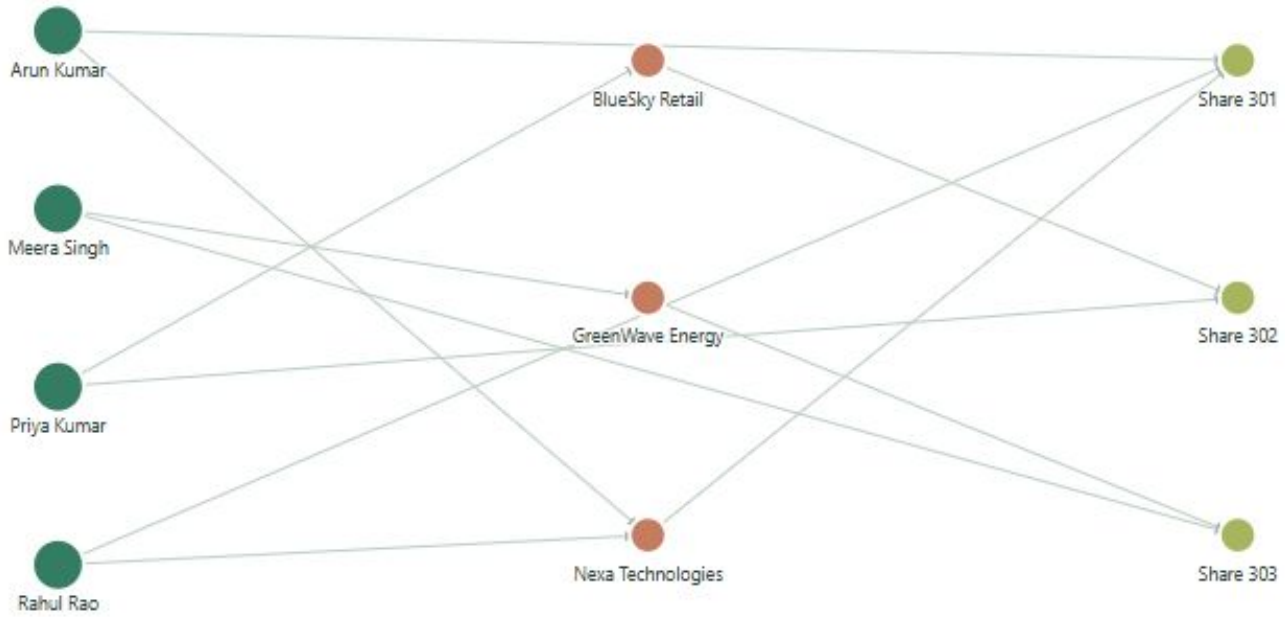


Figure 3. Actual employer and investment graph. User, Company and Share labels are shown in separate columns to make the relationships easier to follow.

Shared investment connections

```
MATCH (u:User)-[:HOLDS]->(s:Share)<-[:HOLDS]-(v:User)
WHERE u.user_id<v.user_id
RETURN u.name AS member_one,v.name AS member_two,collect(s.share_id) AS shared_shares,
count(s) AS shared_count ORDER BY shared_count DESC,member_one,member_two
```

User pair	Shared share
Arun Kumar and Priya Kumar	302
Arun Kumar and Rahul Rao	301

The baseline yields Arun Kumar with Priya Kumar through share 302 and Arun Kumar with Rahul Rao through share 301. The ordering condition returns each user pair once.

Verification and observed results

Integration checks ran against the actual Neo4j instance, not an in-memory substitute. The fixture counts, report results and reconciliation evidence are stored in submission/verification.json. Temporary records are removed after testing.

Verification timestamp 2026-10-20T13:21:18.658729+05:30. Core integration run 11.88 seconds. This duration is a test-run observation, not an Oracle versus Neo4j benchmark.

Check	Outcome
Standalone sample import and reconciliation	PASS
Repeat import preserves data	PASS
Web pages and CSRF protection	PASS
All 15 Cypher reports and graph APIs	PASS
CRUD across all 11 collections	PASS
Validation and restricted deletion	PASS
Ledger lifecycle and permanent audit	PASS
Atomic rollback	PASS
Concurrent overspending protection	PASS
Cleanup and final reconciliation	PASS
Stop and restart persistence	PASS
Browser interaction	PASS

Important cases

The ledger lifecycle test inserts an expense, changes its amount and type, moves it to another account and deletes it. Both account balances return to their initial values while five audit snapshots remain. Invalid references and overspending leave ledger, balance and audit counts unchanged.

The concurrency test submits two INR 700 expenses against an account with INR 980. Exactly one commits, the other is rejected, and INR 280 remains. The temporary transactions and their test audit events are then removed.

A stop and restart check confirms that original users, transactions and balances persist. Browser checks confirm a successful user creation, graph insight selection and node property inspection. All twelve verification groups passed.

Comparison and limitations

Aspect	Oracle DA2	Neo4j DA3
Representation	Relational tables and foreign keys	Labeled nodes and typed relationships
Share ownership	Junction table with quantity	HOLDS relationship with quantity
Query language	SQL and PL/SQL	Cypher pattern matching
Connected questions	Explicit joins or recursive queries	Relationship patterns and paths
Balance and audit	Database trigger	Managed application write transaction
Uniqueness	Primary and unique constraints	Node and relationship key uniqueness
Other integrity rules	Foreign keys and CHECK constraints	Explicit validation in the application
Visualization	Usually rows and report tables	Native graph results and path explorer
Suitability here	Structured ledger and tabular reports	Connected household and investment analysis

Advantages for the selected use case

The graph model makes relationships first class data. Investment quantity belongs naturally to HOLDS, and queries can follow people through households, employers and investments without representing every connection as a separate joined table. Returning paths provides visible explanations for the three connected insights [1].

Limits and tradeoffs

Neo4j Community supports uniqueness constraints, while several existence, type and key constraint features belong to Enterprise Edition [3]. This application therefore validates required fields and relationship cardinalities in Python. Manual Neo4j Browser writes can bypass these checks.

The state lock serializes writes and is designed for a small educational workload. The local administration UI has no user login or authorization. Displayed graphs are bounded to 300 nodes or relationships, and household views to 200 relationships. There is no fraud model, automated investment advice or measured speed advantage over Oracle.

Setup demonstration and references

Reproduce the implementation

On Windows, install Python 3.10 or newer and Ubuntu WSL2. Extract the source ZIP into a writable directory and run Start DA3.cmd. The first run installs Python packages, prepares Podman in Ubuntu and downloads the pinned official Neo4j Community image. It creates private local credentials and imports the sample dataset. Later runs preserve data and work offline.

```
Application     http://127.0.0.1:5001/
Graph explorer  http://127.0.0.1:5001/graph
Neo4j Browser   http://127.0.0.1:7474/
Bolt connection bolt://127.0.0.1:7687
Verify          .venv\Scripts\python.exe verify.py
```

The database uses persistent volume pf-da3-neo4j-data. Stop DA3.cmd stops the application and Neo4j without deleting the volume. Credentials, virtual environments and runtime files are excluded from the submission ZIP.

Demonstration sequence

Show the overview and graph model; add, edit and delete a temporary user; add a 250 rupee expense to account 1001; edit it to 300; delete it and show the restored 66800 balance and permanent audit events. Run G01 to G03 and explain one equivalent SQL query. Use DEMO_GUIDE.md for exact steps and expected results.

Conclusion

The mini project is implemented using a modern graph database with a reproducible dataset, complete record management, atomic ledger behavior and technology specific queries. Neo4j is suitable for the chosen relationship analysis. Oracle remains a strong option for the original relational ledger and its declarative integrity requirements.

References

[1] Neo4j graph database concepts
https://neo4j.com/docs/getting-started/appendix/graphdb-concepts/

[2] Neo4j Python managed transactions
https://neo4j.com/docs/python-manual/current/transactions/

[3] Cypher and Neo4j edition capabilities
https://neo4j.com/docs/cypher-manual/current/introduction/cypher-neo4j/

[4] Neo4j concurrent data access
https://neo4j.com/docs/operations-manual/current/database-internals/concurrent-data-access/

[5] Neo4j official container setup
https://neo4j.com/docs/operations-manual/current/docker/introduction/